

investigation_20260923

**BOB-235 — kinozal live
credential test:
authenticated=False
status='empty' error=' '**

Field	Value
Revision	1
Last modified	2026-09-23T17:50:00Z
Item	BOB-235 (Bug, left open — NOT closed by this investigation)
Method	superpowers:systematic-debugging (root cause before fix)
Secrets	No credential or cookie VALUE appears below (§11.4.10). Only lengths, cookie NAMES, domains, expiry dates, HTTP status codes.

1. Verdict

The item has two separate causes. One is a code defect, now fixed with TDD. The other is on the operator's side: an upstream / environment problem that needs an operator decision.

#	Class	Finding	Disposition
A	Operator / environment: the upstream domain is	The code's hard-coded primary https://kinozal.tv	Operator decision required. See §5.

#	Class	Finding	Disposition
B	dead (not one of the five hypothesised classes)	publishes the A record 127.0.0.1 . Both Cloudflare and Google DNS-over-HTTPS return it, so no local resolver is intercepting. The kinozal leg never reaches the tracker, and every run fails with a connection refused.	
	Code defect: an empty result was reported where the tracker was actually unreachable (a false-null)	In <code>_search_kinozal</code> , the except Exception: branch only logged the error. It stashed no diagnostic, so the chip read <code>status=empty</code> , <code>error=None</code> . The BOB-172/BOB-178 fixes left this exception leg open.	Fixed with TDD. See §4.
	Blocker behind A (a Cloudflare check that a script cannot pass)	The live mirror <code>kinozal.guru</code> answers every non-browser HTTP client with a Cloudflare “Just a moment” HTTP 403 . This happens even	Operator decision required. Fixing A alone will NOT make the test pass.
C			

#	Class	Finding	Disposition
		when the request carries the operator's valid uid/pass session cookies.	

Hypotheses ruled out, with evidence: - **Env not reaching the container:** ruled out. All three KINOZAL_* variables are present and non-empty (§3.3). - **Login-form or parse change:** ruled out. The code never reached kinozal, so no HTML was parsed at all. - **Credentials expired: UNKNOWN.** No request has reached a real kinozal login endpoint since kinozal.tv died. kinozal.guru answers the login POST with a Cloudflare 403 before any authentication runs. The operator's browser cookies for .kinozal.guru are **not** expired (§3.5). - **CAPTCHA / bot protection:** this is real, but it applies to the *mirror* (C). It is not what failed this run. This run failed on (A).

2. Reproduction (RED on the live stack, unchanged, no restart)

```
$ nice -n 19 .venv/bin/python -m pytest "tests/integration/
test_tracker_auth_live.py::test_private_tracker_credentials_authent
-v --import-mode=importlib -p no:cacheprovider
E   Failed: kinozal: authenticated=False (expected True) – the
STORED credentials failed to log in. status='empty', error=''.
This is a genuine bad/expired-credential failure, not a transient.
FAILED tests/integration/
test_tracker_auth_live.py::test_private_tracker_credentials_authent
===== 1 failed in 21.76s
=====
```

Container log (qbittorrent-proxy) for the same search, kinozal lines only:

```
2026-09-23 17:35:49,806 - ERROR - Kinozal search error: Cannot
connect to host kinozal.tv:443 ssl:default [Connect call failed
('127.0.0.1', 443)]
2026-09-23 17:35:49,806 - INFO - Tracker kinozal: 0 results for
'ubuntu'
```

3. Evidence

3.1 Code path → why error= ''

download-proxy/src/merge_service/search.py::_search_kinozal: - It uses `base_url = os.getenv("KINOZAL_MIRRORS", "https://kinozal.tv")`. `KINOZAL_MIRRORS` has length 0 in the container, so the base URL is `kinozal.tv`. - The `aiohttp` connect failure lands in except `Exception` as `e: logger.error(...)`. Before the fix, that branch stashed nothing in `_last_public_tracker_diag`. - `_search_one` then sets `status = "empty"` (because there are no results). With no diagnostic to read, error stays `None`, which the test renders as `''`. - `_refresh_stat_auth_state` returns `False` because no `_tracker_sessions["kinozal"]` entry was ever written. That entry is written only after a successful search fetch.

3.2 DNS: the primary domain resolves to loopback, and not only locally

```
== host getent
127.0.0.1      STREAM kinozal.tv
== container
127.0.0.1      kinozal.tv kinozal.tv      (no /etc/hosts
entry; resolv.conf -> 127.0.0.53)
== host resolvectl: kinozal.tv: 127.0.0.1 -- link: enp5s0
== DoH cloudflare: {"Answer":
[{"name":"kinozal.tv","type":1,"TTL":1851,"data":"127.0.0.1"}]}
== DoH google:      {"Answer":
[{"name":"kinozal.tv","type":1,"TTL":1571,"data":"127.0.0.1"}]}
== NS (DoH google): ns-uk.topdns.com., ns-usa.topdns.com., ...
== control needle rutracker.org: 104.21.32.39 (host) /
2606:4700:3037::ac43:b6c4 (container)
```

The control needle shows that the same resolver path does resolve a real tracker, so this is not a blind instrument (§11.4.273). Two independent DoH resolvers return 127.0.0.1, which means the published authoritative record is loopback. Local interception is ruled out.

Mirrors (from the `trackers.py` roster ("`kinozal.tv`", "`kinozal.me`", "`kinozal.guru`")):

```
kinozal.me  DoH -> 104.21.87.84 / 172.67.142.133 ; host resolver
-> 79.101.65.65, TLS cert fails verification (unable to get local
issuer certificate) -> curl 000
kinozal.guru DoH -> Cloudflare (104.21.25.173 / 172.67.134.107) ;
host resolver agrees
```

```
https://kinozal.tv/ -> 000 (refused)
https://kinozal.guru/ -> 302 -> /login.php?m=5 (the live site)
```

UNCONFIRMED: why the host resolves kinozal.me to 79.101.65.65 with an untrusted cert. A local/ISP-level redirect is one candidate, but it is not proven here. kinozal.me is therefore not a usable mirror from this host as measured.

3.3 Env reaches the container (names + lengths only)

```
podman exec qbittorrent-proxy env | cut -d= -f1 | grep -i kinozal
KINOZAL_COOKIES KINOZAL_PASSWORD KINOZAL_USERNAME
KINOZAL_USERNAME len=12 KINOZAL_PASSWORD len=18
KINOZAL_COOKIES len=95 KINOZAL_MIRRORS len=0
```

3.4 The live mirror sits behind a Cloudflare check that scripts cannot pass

From inside the container, printing only statuses and cookie names:

```
POST https://kinozal.guru/takelogin.php (stored creds) ->
login_status 403, cookie_names []
GET https://kinozal.tv/... -> URLError
[Errno 111] Connection refused
GET kinozal.guru/browse.php anon (host curl) -> 403, body:
"Just a moment", "challenge-platform", "cloudflare"
GET kinozal.guru/browse.php + KINOZAL_COOKIES, 2 UAs -> status
403, rows 0, cf_challenge True, logout False (both)
GET kinozal.guru/browse.php via aiohttp (the real client) ->
status 403, cf True
```

3.5 The cookie file: presence, age, mode, and kinozal cookie NAMES only

```
-rw----- 1 milosvasic milosvasic 90258 2026-09-22 10:05:00
+0200 /home/milosvasic/Downloads/cookies_kinozal.txt
.kinozal.guru uid expires=2027-09-19T11:57:18Z
.kinozal.guru pass expires=2027-09-19T11:57:18Z
.kinozal.guru u_eb3299ed2c
expires=2026-08-15T22:00:00Z (expired)
.kinozal.guru eb3299ed2c_blockTimer
expires=2026-08-15T12:07:17Z (expired)
.kinozal.guru eb3299ed2c_delayCount
expires=2026-08-15T12:07:21Z (expired)
(no cf_clearance cookie for any kinozal domain)
env KINOZAL_COOKIES cookie names: eb3299ed2c_blockTimer
u_eb3299ed2c uid pass eb3299ed2c_delayCount
```

Findings: - The file is mode 0600, but it is a **whole-browser export**: 643 cookie rows across many unrelated domains (yandex, rutracker, github, semgrep, ...). Only 5 rows are kinozal. scripts/load-tracker-cookies.sh filtered correctly: the container var contains only the kinozal names. Even so, it is worth the operator's attention that a full browser jar sits under the kinozal filename. - The operator's session is scoped to **.kinozal.guru**, which confirms that kinozal.guru is the domain the operator actually uses. - **No code consumes KINOZAL_COOKIES**. `grep -rn KINOZAL_COOKIES download-proxy/src plugins scripts/load-tracker-cookies.sh` returns 0 matches. The autoload puts the variable into the container, and nothing reads it. The kinozal leg only does a username/password POST. Even if the variable were consumed, §3.4 shows the cookies alone do not pass the Cloudflare check. - The test's ubuntu query was unrelated to all of this. The failure happens before any search is sent.

4. Code fix (strict TDD) for defect B

Files touched: - download-proxy/src/merge_service/search.py: `_search_kinozal`'s except branch now stashes `_last_public_tracker_diag["kinozal"] = {error_type: <exception class>, error: "Kinozal request failed: <msg>", ...}`. This reuses the orchestrator's own exception format (`_search_one`) and adds no new vocabulary. It deliberately does not call `_classify_plugin_stderr`: the message contains `ssl:`, so that classifier would mislabel it as `tls_failure`. - NEW tests/unit/merge_service/test_bob235_kinozal_connect_failure_diagnostic.py: 2 RED/GREEN tests (the diagnostic is set; it is not mislabelled as TLS) plus a negative control (a healthy round trip stashes no diagnostic and records a session).

RED (pre-fix code):

```
FAILED ...::TestKinozalTransportFailureSetsDiagnostic::test_diagnostic
FAILED ...::TestKinozalTransportFailureSetsDiagnostic::test_connect
>         assert diag is not None, "a kinozal transport failure must
not be reported as a silent empty result (BOB-235)"
2 failed, 1 passed in 0.42s
```

GREEN (post-fix):

```
bob235 + bob178 + bob177 + test_private_tracker_search: 48 passed
in 2.78s
tests/unit/merge_service (full): 1092 passed, 1 warning in 140.04s
tests/unit/test_merge_trackers.py +
```

test_private_tracker_search.py: 49 passed in 3.89s
ruff check (both files): All checks passed!

Effect of the fix, once the container picks it up: the kinozal chip will read

status=error, error_type=ClientConnectorError, error="Kinozal request failed: Cannot connect to host kinozal.tv:443 ..." instead of the false-null. The live test will **still FAIL**, correctly, because kinozal cannot be reached. The message contains none of the test's transient markers, so it is not converted into a skip.

Not live yet. Per the task constraints, the stack was not restarted. The fix reaches the running service only after `./start.sh --reload-python`, which is an operator or conductor step.

Mirror sites not changed (outside this item's scope, recorded honestly): the rutracker, nnmclub and iptorrents search legs may have the same log-only except shape. They were not audited here. A shared helper would follow the §11.4.251 pattern that BOB-177 already applied to the HTTP-status guard.

5. Operator action required (nothing below was done by the agent)

1. **Choose kinozal's reachable domain.** kinozal.tv publishes 127.0.0.1. Either:
 - 1. set `KINOZAL_MIRRORS=https://kinozal.guru` in `.env`, then run `./start.sh --recreate`; or
 - (b) decide that the `trackers.py` roster primary should change from `kinozal.tv` to `kinozal.guru`. That code change also affects `search.py:1258`, `plugins/kinozal.py:157` and `.env.example:82`.**(a) alone does not fix the test**, because of step 2.
2. **Decide on a Cloudflare challenge strategy for kinozal.guru.**
Every non-browser client gets a 403 "Just a moment" page, even with a valid session. Options:
 - 1. run a challenge solver (e.g. FlareSolverr; it is referenced only in `NO_PROXY`, and no such service exists);
 - (ii) export a `cf_clearance` cookie from the browser and make `_search_kinozal` consume `KINOZAL_COOKIES`. Such a cookie is bound to the user-agent and IP address and expires quickly, and the code currently ignores that variable;
 - (iii) route kinozal through Jackett's kinozal indexer;
 - (iv) accept kinozal as operator-blocked and have the live test skip on a recognised challenge marker. That would need the

diagnostic to carry a `upstream_captcha-class` token for a Cloudflare 403, which is a separate change.

Which option is right is an operator decision (§11.4.66).

3. **Credential validity stays UNKNOWN** until step 1 and step 2 get a request past the challenge to `takelogin.php`. Do not rotate the credentials based on this failure.
4. (Hygiene, optional) Re-export `~/Downloads/cookies_kinozal.txt` with only the kinozal cookies. The current file is a full browser jar (643 rows, including rutracker/yandex/github sessions).

6. Honest gaps

- Whether the stored kinozal username/password still work: **UNKNOWN** (see §5.3).
- When `kinozal.tv` began publishing 127.0.0.1: **UNKNOWN**. The Issues history was not searched for a last-known-good kinozal live pass.
- Why `kinozal.me` resolves locally to a different IP with an untrusted cert: **UNCONFIRMED** (§3.2).
- The fix has unit evidence only. Runtime evidence on the live container is **PENDING** a `--reload-python` (not performed, per constraints).